

常见错误:

错误 1: 路径名或者变量有空格时候, 会造成参数太多的错误提示

错误2: Git Bash 中的路径切换为正斜杠 "/" , 而不是 Windows 中的反斜杠 "\"

错误3: 使用某些查找命令时, 文本过长 Git 会自动进入浏览模式, 此时无法再输入任何指令

错误4: 向 github 推送或者下载代码时网络超时

错误5: windows 下解决Git报错: LF will be replaced by CRLF the next time Git touches it

基本 Git 命令行:

Git分区:

分支:

分支Git命令行:

代码托管 (github) :

补充: GitBash / Mac 常见指令:

变基 rebase:

应用场景1: 整合非重要版本记录

应用场景2: 避免拉取远程代码时 Git 中产生分叉

注意事项:

协同开发:

项目实践:

开源精神:

配置文件:

Git 忽略文件:

任务管理:

常见错误:

错误 1: 路径名或者变量有空格时候, 会造成参数太多的错误提示

```
bash: cd: too many arguments
```

解决方案: 使用双引号将带有空格的部分括住

错误2: Git Bash 中的路径切换为正斜杠 "/" , 而不是 Windows 中的反斜杠 "\"

在 Git Bash 中切换路径可以使用 `cd` 命令, 具体步骤如下:

1. 打开 Git Bash 终端;
2. 使用 `cd` 命令加上要切换的路径, 例如: `cd /c/Users/username/Documents;`
3. 按下回车键, 即可切换到指定的路径。

需要注意的是, 路径中需要使用正斜杠 / 分隔路径, 而不是反斜杠 \。此外, 在 windows 系统中, 你可以使用 `cd` 命令加上盘符来切换到对应的磁盘, 例如: `cd /c` 切换到 C 盘。

正确示例:

```
1 Users MINGW64 /d
2 $ cd D:/"yi yi"/文档合集/新技能新气象/计算机/Git实践案例
3 Users MINGW64 /d/yi yi/文档合集/新技能新气象/计算机/Git实践案例
```

错误3：使用某些查找命令时，文本过长 Git 会自动进入浏览模式，此时无法再输入任何指令

```
1 | commit version
2 | Author: User's name <email>
3 | Date:    Sun Jul 21 21:31:06 2024 +0800
4 |
5 |     v1
6 | (END)
```

解决方案：英文键盘 Q 退出

错误4：向 github 推送或者下载代码时网络超时

git clone 或 git pull 等操作时出现 "fatal: unable to access ' <https://github.com/.../.git> ': Recv failure Connection was reset "的报错。这个问题通常是由网络连接问题或代理设置不正确导致的。

解决方案：通过在终端执行以下命令，可以取消 Git 的代理设置：

```
1 | git config --global --unset http.proxy
2 | git config --global --unset https.proxy
```

错误5：windows 下解决Git报错： LF will be replaced by CRLF the next time Git touches it

原因：

Unix/Linux 采用换行符LF表示下一行（LF：LineFeed），即：\n；

Dos 和 Windows 采用回车+换行CRLF表示下一行（CRLF：CarriageReturn LineFeed），即：\r\n；

Mac OS采用回车CR表示下一行（CR：CarriageReturn），即：\r。

解决方案：

```
git config --global core.autocrlf true
```

基本 Git 命令行：

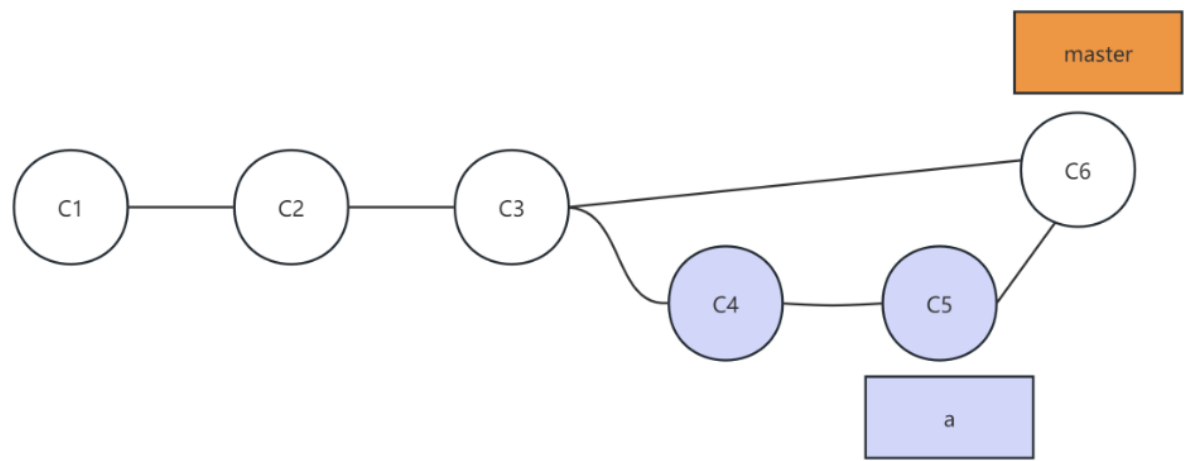
指令	作用
git init	初始化当前路径下的文件夹，开启 Git 管理
git status	显示当前文件夹中所有文件
git add 文件名(test.txt) git add .	将指定文件管理权限交于 Git 将当前文件夹中所有文件管理权限交于 Git
git commit -m 'v1'	将当前 Git 所管理的文件 ¹ 生成新的版本（其中 '' 中为描述信息）
git log git log --graph git log --graph --pretty=format:"%h %s"	查看版本历史提交记录 以图形化方式查看版本历史提交记录 以格式化流程查看版本历史提交记录
git reset --hard 版本号	回滚到对应版本（此时文件夹中的内容也会相应改变）
git reflog	查看历史所有版本（包括回滚版本之后的版本）
git checkout 文件名(test.txt) git reset HEAD 文件名(test.txt)	将工作区的未管文件转化为已管文件（通过撤销修改或删除新文件实现） 将暂存区文件转化为工作区的未管文件（即 add 命令的逆操作）

Git分区：

区域	文件类型
工作区（文件夹）	已管文件 未管文件[^2]（新文件、修改文件）
暂存处	通过add命令提交的文件（已管文件）即均存储在暂存区中
版本库	通过commit命令将暂存区中的已管文件所生成的版本存储在版本库中

分支：

☀ Git 管理生成的不同版本之间并不是独立的，新版本实际上只会保留新修改的文件，而其余文件则通过指针指向之前版本。



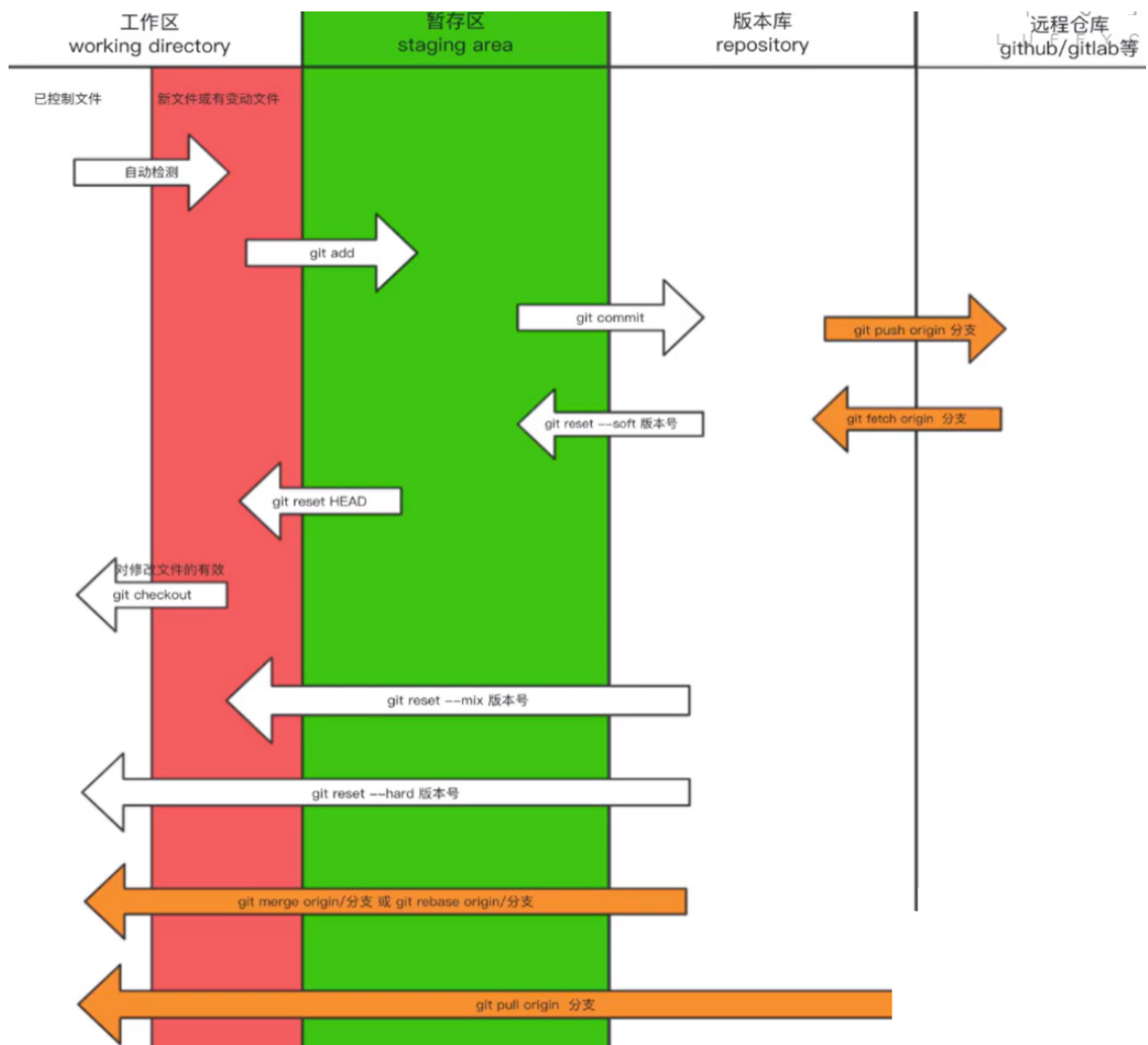
- 通过分支即可以实现项目 bug 的紧急修复：
- 1.将当前正在开发但未完成的版本形成新分支（dev）
 - 2.修改 bug 的版本也形成新分支（bug）
 - 3.完成后合并到主干 master

分支Git命令行：

指令	作用
git branch	显示目前所处分支 ^[^3]
git branch 分支名	创建新分支（指向当前所处分支的当前版本，而非最新版本）
git checkout 分支名	跳转到对应分支
git branch -d 分支名	删除对应分支
git merge 分支名	合并 ^[^4] 对应分支的对应版本到当前分支的当前版本

代码托管（github）：

指令	作用
git remote add 别名 远程仓库地址	为远程仓库起别名 ^[^5]
git push -u 别名 分支名	向远程仓库推送代码
git clone 远程仓库地址	首次拷贝远程仓库代码 ^[^6]
git pull 别名 分支名	选择性更新本地代码



补充：GitBash / Mac 常见指令：

指令	作用
cat 文件名	查看文件内容
cd ..	回退上一级目录
ls	浏览当前文件夹
pwd	显示当前目录的路径名
mkdir 目录名	新建目录
rmdir 目录名	删除目录
rm 文件名	删除文件
file 文件名	显示文件类型
open 文件名	使用默认的程序打开文件
open .	打开当前目录
date	显示系统的当前日期和时间
clear	清除屏幕或窗口内容
whoami	显示当前正进行操作的用户名

变基 rebase:

应用场景1：整合非重要版本记录

指令	作用
git rebase -i 版本号	将当前版本与指定版本之间的所有版本合并
git rebase -i HEAD~num	从当前版本（包含自身）向前合并 num 个版本

```
1 //执行完以上任意一个操作后,修改合并方式, :wq 保存
2 pick 0aa4591 v2 ---> pick 0aa4591 v2
3 pick 9few83b v3 ---> s 9few83b v3
4 pick 3jnk30f v4 ---> s 3jnk30f v4
```

```
1 //合并之后,调整新记录的信息(替换为步骤三代码), :wq 保存
2 # This is the 1st commit message:
3 v2
4 # This is the commit message #2:
5 v3
6 # This is the commit message #3:
7 v4
```

```
1 # This is the 1st commit message:
2 v2 & v3 & v4
```

☀通常 rebase 合并操作在提交代码至远程仓库之前, 若先提交至远程仓库再合并记录会导致版本记录混乱

应用场景2：避免拉取远程代码时 Git 中产生分叉

指令	作用
git fetch 别名 分支名	将远程仓库中的分支拉取到本地仓库
git rebase 别名/分支名	合并两条分支（无分叉）

```
1 git pull origin dev (替换为步骤二代码)
```

```
1 git fetch origin dev
2 git rebase origin/dev (提前切换到dev分支)
```

注意事项:

- 1、如果在 rebase 合并时产生冲突, 自行解决冲突之后, 将新代码添加至暂存区或者提交为新版本并执行以下命令即可继续正常运行:
'git rebase --continue '
- 2、下载 Beyond Compare 软件（正版需要收费）可用于自动解决冲突问题:

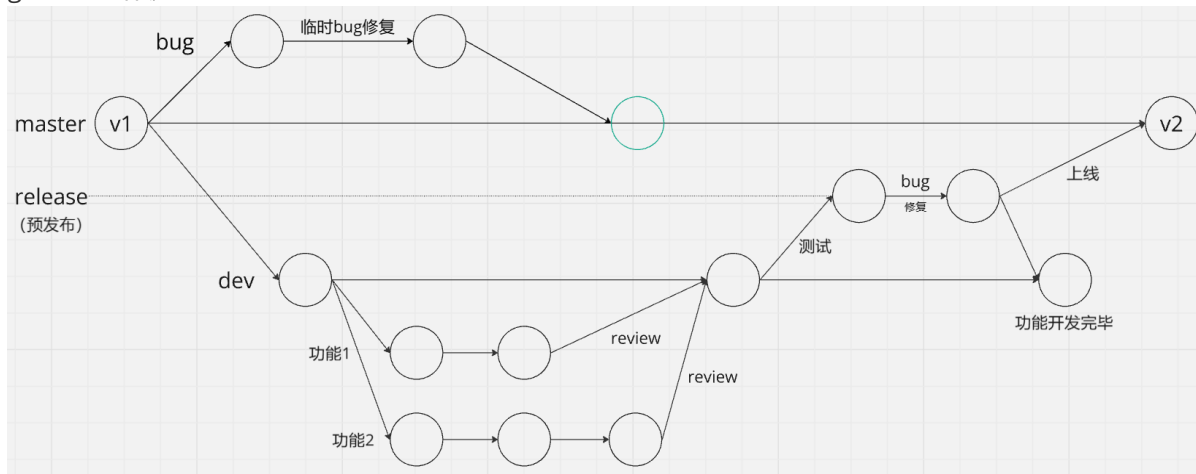
```

1 git 配置:
2 git config --global merge.tool bc4
3 git config --global mergetool.bc4.cmd '"D:\应用合集\BCompare\Beyond Compare
5\BComp.exe" "$LOCAL" "$REMOTE"'
4 git config --global mergetool.keepBackup false
5 应用 Beyond Compare 解决冲突:
6 git merge dev
7 git mergetool

```

协同开发:

gitflow 工作流:



项目实践:

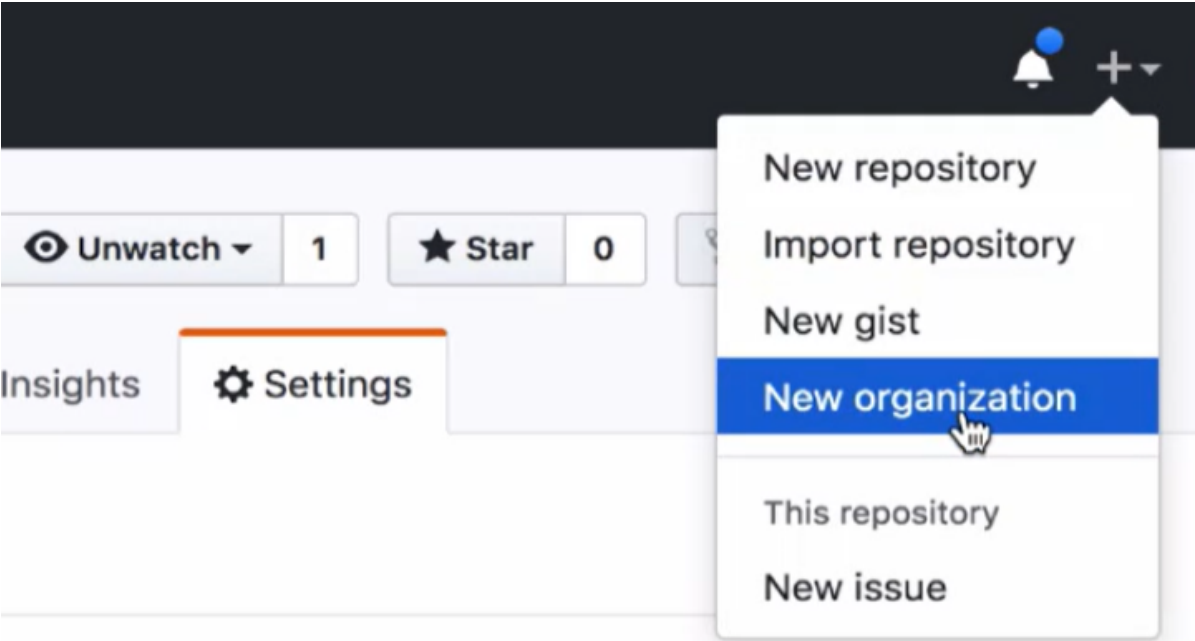
1.创建 (master) 项目以及组织

邀请同伴 (个人项目): github -> Settings -> Collaboration

Options	Collaborators
Collaborators	Push access to the repository
Webhooks	Awaiting peiqiwu's response
Notifications	Copy invite link
Integrations & services	Cancel invite
Deploy keys	Search by username, full name or email address
	You'll only be able to find a GitHub user by their email address if they've chosen to list it publicly. Otherwise, use their username instead.
	Add collaborator

邀请同伴 (公司项目): New organization

组织中可以有多个项目，而组织中的成员可以对组织中的某些项目进行读、写操作



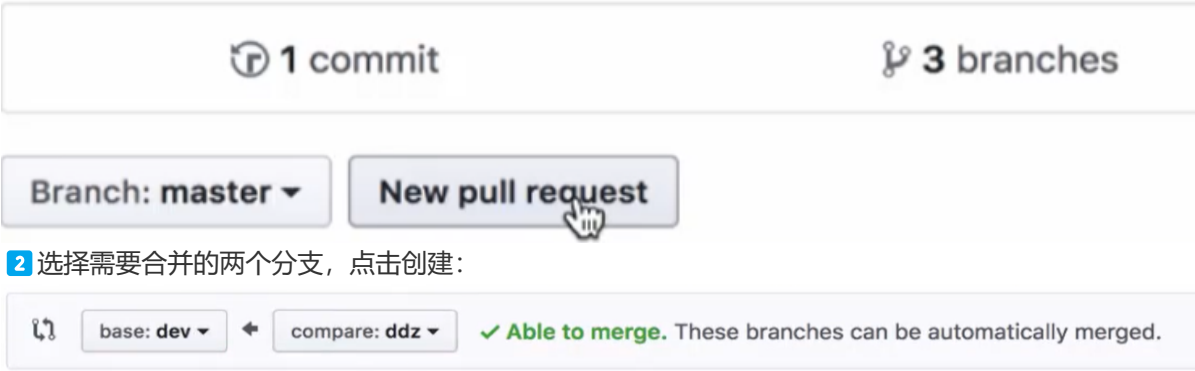
补充：为版本打标签以便后续管理

指令	作用
git tag -a 版本标记 -m '补充信息'	为不同的版本打标签
git push origin --tags	将标签推送至 github

- 2.创建分支（bug、release、dev）并上传到 github
 - 3.设置组织成员权限并将相应的成员拉入相应项目（Settings -> Collaboration）
 - 4.成员创建各自分支并进行开发
 - 5.成员开发完毕后进行代码 review
- 方法：Settings -> Branches -> Add rule -> Require pull request reviews before merging
- 效果：通过设立规则，要求每次将代码合并到 dev 分支前，需要指定人员进行 review
- review步骤：

1 成员需要在 github 中选择 New pull request 提醒项目负责人进行代码 review

东北热



2 选择需要合并的两个分支，点击创建：

3 负责人进行 review（右上角）后同意合并（左下角）：

 **Review required** [Add your review](#)
At least 1 approving review is required by reviewers with write access. [Learn more.](#)

 **Merging is blocked**
Merging can be performed automatically with 1 approving review.

As an administrator, you may still merge this pull request.

Merge pull request You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

☒ **Use your administrator privileges to merge this pull request.**

Confirm merge

Cancel

4 负责人同意之后，分支会进行自动合并，无需成员再合并，此时还可以在 github 中将分支快速删除

Pull request successfully merged and closed

You're all set—the `ddz` branch can be safely deleted.

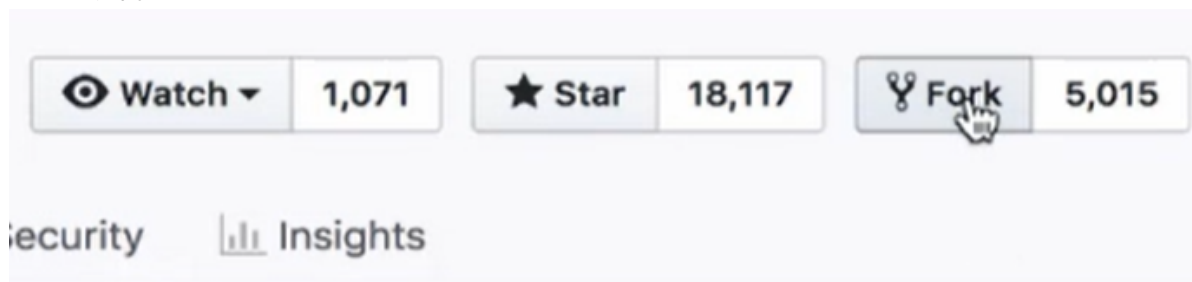
Delete branch

6.代码 review 完毕之后，进行线上测试并提交上线

开源精神：

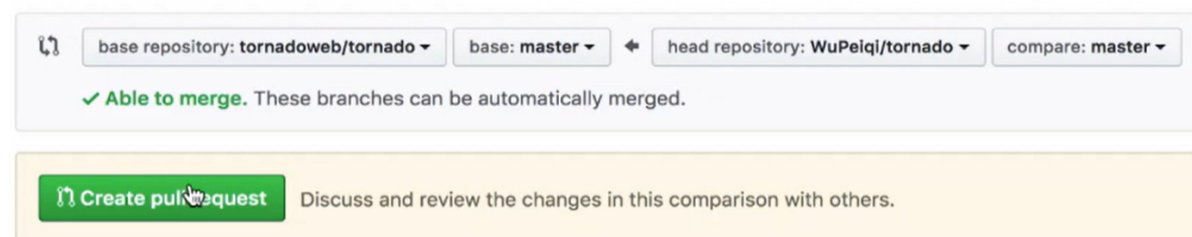
在闲暇时间为其他开源项目提供修改意见或者代码，何尝不是未来的一件谈资呢？

1.Fork 源代码



2.本地进行开发或修改

3.将修改版本提交给原作者 (New pull request)



配置文件：

```
1 项目配置文件：项目/.git/config
2  git config --local user.name ''
3  git config --local user.email ''
4  项目配置文件： ~/.git/config
5  git config --global user.name ''
6  git config --global user.email ''
7  系统配置文件： /etc/.git/config
8  git config --system user.name ''
9  git config --system user.email ''
10 注意：修改系统配置文件需要有 root 权限
```

🔦 配置文件的应用优先级为：项目配置文件 > 项目配置文件 > 系统配置文件（即就近原则）

Git 忽略文件：

有时我们希望 Git 管理的文件夹中有部分文件脱离管理，此时可以使用 .gitignore 文件来管理：

- 1.在 Git 管理的文件夹下创建 .gitignore 文件
- 2.将需要忽略的文件（包括自身）写入 .gitignore 文件

```
1  *.h（其中 * 为任意字符）
2  !a.h（结合上命令表示将除 a.h 外所有以 .h 结尾的文件忽略）
3  *.py[c|cpp|text]（表示忽略以 .py/c/cpp/text 结尾的文件）
4  文件夹/（表示忽略文件夹下的所有文件，也可以配合！使用）
5  .gitignore
```

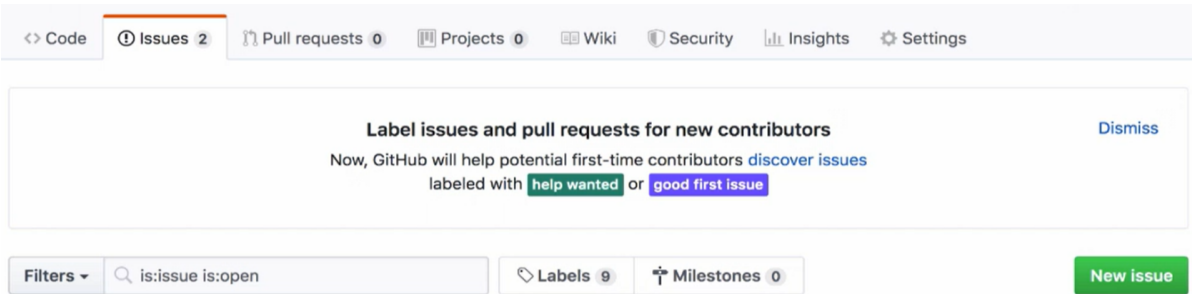
The screenshot shows the GitHub repository page for 'gitignore' by 'wirecat'. The repository is public and has 3366 watches, 83.2k forks, and 159.9k stars. It contains 8 branches and 0 tags. The repository description is 'A collection of useful .gitignore templates'. The repository has 3,588 commits and a recent merge pull request #3997 from 417-72KI/update-objc. The repository includes a README, CC0-1.0 license, Code of conduct, Security policy, Activity, Custom properties, 160k stars, 3.4k watching, 83.2k forks, and a Report repository link. The repository also has a Releases section with no releases published.

File	Description	Time
.github	Adds a relationship question	3 years ago
Global	Merge pull request #4435 from zengxs/patch-1	2 months ago
community	Merge pull request #4448 from DrKeek/community_UIPath	2 months ago
AL.gitignore	Add .gitignore for Microsoft Business Central	3 years ago
Actionsript.gitignore	Fix comments on same line causing ignore to break	7 years ago
Ada.gitignore	ensure single trailing newline	10 years ago
Agda.gitignore	Add MALonzo directory. (#2978)	5 years ago
Android.gitignore	Android Studio	2 years ago
AppEngine.gitignore	Added template for Google App Engine	10 years ago
AppceleratorTitanium.gitignore	Added Appcelerator Titanium .gitignore file.	12 years ago

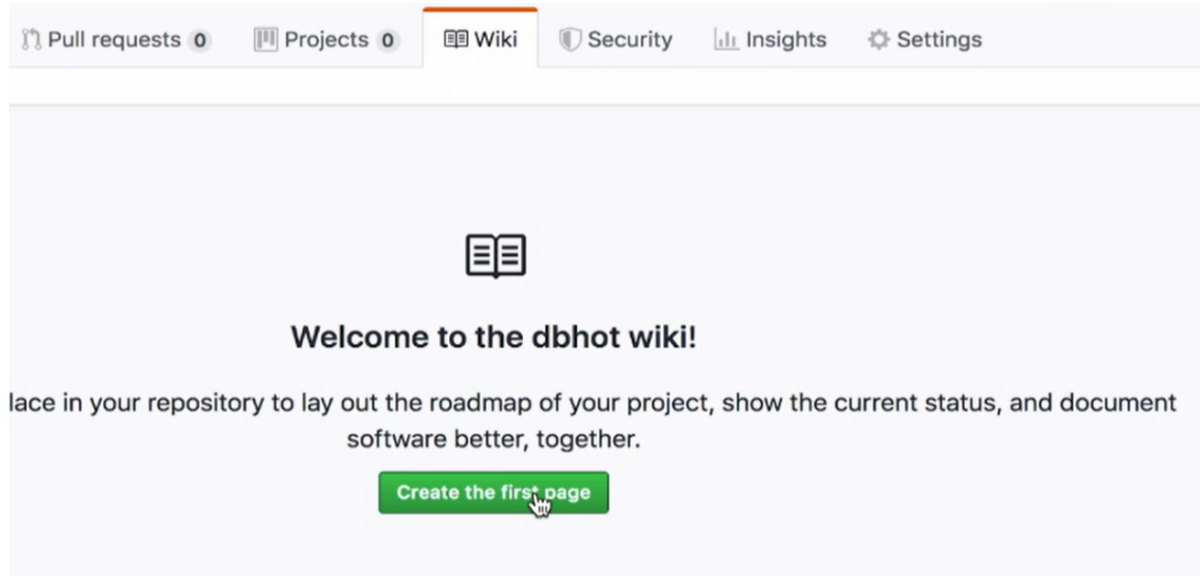
🔦 常见的建议可忽略文件在 github 也有总结，可自行下载

任务管理：

github -> Issues 面板可以进行问题提问与讨论：



github -> Wiki 面板进行项目详细介绍或者注意事项等等，以便后续新人快速上手项目



持续更新中... 🔄

1：在git commit -m "时出现命令执行失败的原因是没有在本地设置用户名和邮箱，在命令行设置一下即可：

```
1 | git config --global user.email 'you@example.com'
2 | git config --global user.name 'Your name'
```

[^2]：已管文件与未管文件之间由 Git 进行自动检测与转换

[^3]：主分支名称为 master，其余分支名称自拟。正式的开发至少有两个分支（master 与 dev），master分支只存储正式版本，而 dev 分支用于开发新功能

[^4]：在合并不同的分支时，有可能会因为文件冲突而需要手动修复

```
1 | 例如：
2 | <<<<<< HEAD
3 | =====
4 | >>>>>> dev
```

[^5]：一个项目仓库只需要在其初始化的时候起一次别名即可，后续可直接使用该别名

[^6]: 此时将远程仓库中的所有分支均拷贝下来，只不过在 Git 中没有直接显示，不过依旧可以使用 git branch 指令查看并操作

注意：上传代码时每次上传一个分支，拷贝代码时只需要拷贝一次

1. [ee](#)